

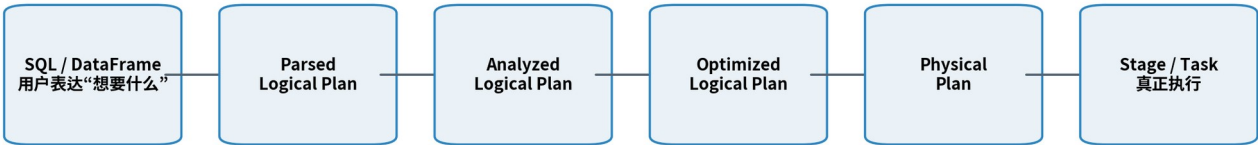
Spark 原理详解 - 第 16 讲

Catalyst Optimizer 与执行计划：Spark 是怎样“看懂并改写”你的 SQL 的

适用对象：已经完成 RDD、Shuffle、DataFrame / Spark SQL 基础，希望真正看懂 explain() 与 Spark SQL 执行过程的学习者。

本讲目标：学完以后，你应该能够把一条 DataFrame/SQL 语句沿着“逻辑计划 -> 优化逻辑计划 -> 物理计划 -> Stage/Task”追踪下去，并能在 explain() 中识别 Scan、Filter、Project、Exchange、Aggregate、BroadcastHashJoin、SortMergeJoin、AdaptiveSparkPlan 等关键节点。

本讲不是要求你背 Catalyst 的内部 Scala 类，而是形成一个能够用于读执行计划、排查慢 SQL、理解 Spark 自动优化的最低闭环能力。



Catalyst 主要工作在“逻辑计划 -> 优化逻辑计划 -> 物理计划选择”这一段

本讲知识层	你要回答的问题
逻辑层	用户写的 SQL / DataFrame 到底表达了什么？
解析与分析	表名、列名、类型能否解析成确定含义？
优化层	哪些等价变换能减少无效工作？
物理层	同一个逻辑目标究竟选哪种执行算法？
执行层	Physical Plan 怎样进一步变成 Stage 与 Task？

1. 为什么 DataFrame 能被自动优化，而 RDD 很难？

上一讲我们已经知道：DataFrame 不只保存“数据”，还保存 Schema 和结构化表达式。这个差异决定了 Spark 能否理解你的计算。

RDD 写法	DataFrame / SQL 写法
<code>rdd.filter(lambda x: x[3] == "Beijing")</code>	<code>df.filter(df.city == "Beijing")</code>
Spark 主要看到：调用一个用户函数	Spark 能看到：对 city 列做等值过滤
函数内部语义对优化器近似黑盒	表达式结构可被分析和改写
优化空间较小	可以做谓词下推、列裁剪、Join 选择等

核心原理：要自动优化，系统首先必须“理解”计算语义。DataFrame / SQL 把计算表达成结构化表达式，因此 Spark 可以在真正执行前重写计划。

一个直观比喻：RDD 更像你把一段已经写好的“黑盒程序”交给工人；DataFrame 更像你把“施工图纸”交给总工程师。总工程师可以重新安排施工顺序，只要最终结果等价。

2. Catalyst 到底是什么？

Catalyst 可以理解为 Spark SQL 的查询优化框架。你写 SQL 或 DataFrame API 后，Spark 不会马上逐行执行，而是先构建查询计划，再进行分析、规则优化和物理执行策略选择。

不要误解：Catalyst 不是一个单独运行的“AI 优化器”，也不是把所有 SQL 都变成理论最优计划。它依赖规则、统计信息、代价信息以及运行时信息来选择和调整执行方式。

3. 一条 SQL 从文字到执行：五个关键阶段

```
query = (orders
    .filter("status = 'paid'")
    .groupBy("city")
    .sum("amount"))

query.explain(mode="extended")
```

这段代码表面上只有过滤和分组求和，但 Spark 内部会依次构造多个层次的计划。

阶段	最通俗的理解	典型任务
Parsed Logical Plan	“句子大概是什么意思”	解析语法结构
Analyzed Logical Plan	“这些表和列到底指谁”	解析列、类型、函数、关系
Optimized Logical Plan	“不改变答案的前提下重排步骤”	过滤下推、列裁剪、常量折叠等
Physical Plan	“具体用哪种算法跑”	选择 HashAggregate、Join 策略、Exchange 等
Stage / Task	“真正分给集群干活”	按 Shuffle 边界切 Stage，生成 Task

记忆方法：Logical Plan 主要回答“算什么”；Physical Plan 主要回答“怎么跑”。Stage/Task 才是集群真正执行的工作单元。

4. Parsed Logical Plan：先把代码变成一棵“语义树”

解析阶段关注的是结构，而不是最终执行算法。比如 SQL：

```
SELECT city, SUM(amount) FROM orders WHERE status='paid' GROUP BY city
```

Spark 首先可以形成类似下面的树形含义：

```
Aggregate [city], [city, SUM(amount)]
├── Filter status = 'paid'
│   └── Relation orders
```

此时有些列可能仍以“未解析属性”的形式存在，因为 Spark 还需要去 Catalog / Schema 中确认 orders 是否存在、city 是哪一列、amount 是什么类型。

初学者易错：Parsed 并不等于“已经知道怎么执行”。它只是把文本或 API 表达式组织成结构。

5. Analyzed Logical Plan：把名字绑定到真实对象

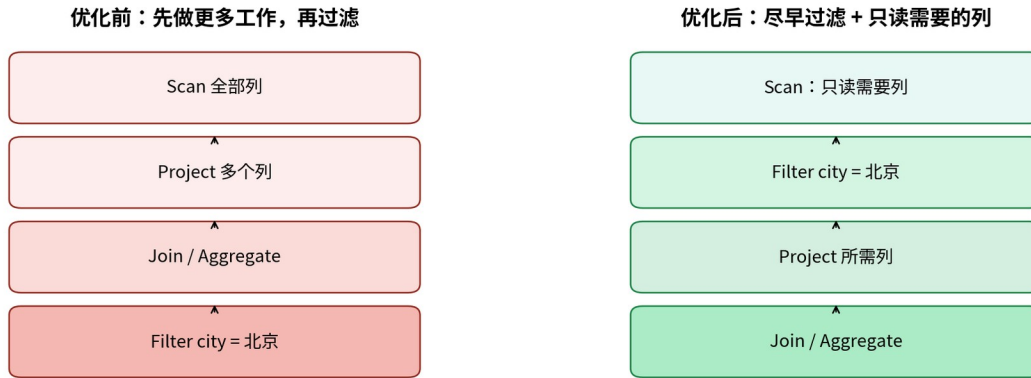
分析阶段会结合 Catalog、Schema、函数注册信息等完成解析。例如：

- 确认 orders 表或 DataFrame 的关系来源。
- 确认 city、status、amount 列是否存在。
- 确认 amount 能否进行 SUM。
- 必要时插入类型转换。

如果你写了不存在的列名，很多错误就是在分析阶段暴露，而不是等到 Executor 真正扫描几 TB 数据以后才发现。

6. Optimized Logical Plan : Catalyst 真正开始“改写”

优化的目标不是改变你的业务含义，而是在等价前提下尽量减少数据量、计算量和不必要的操作。最值得初学者掌握的是下面几类。



核心目标：减少被读取、被传输、被参与后续算子的无用数据

6.1 Predicate Pushdown / Filter Pushdown（谓词下推）

如果能够把过滤条件推到数据源附近，就尽量早过滤。例如 1 TB 订单中只有 2% 是北京订单，那么“先过滤再 Join”往往比“Join 完再过滤”便宜得多。

```
df.filter(df.city == 'Beijing').select('product_id', 'amount')
```

底层原因：越早丢掉无用行，后面的网络 Shuffle、Join、Aggregate 就越少处理数据。

6.2 Column Pruning（列裁剪）

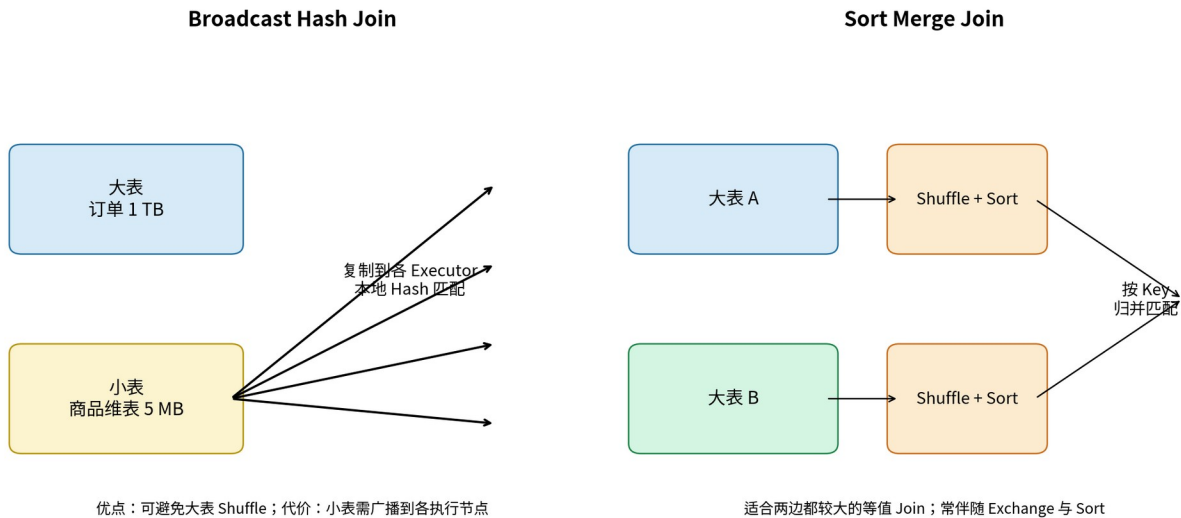
如果一个 Parquet 表有 100 列，而你的查询只需要 city 和 amount，优化器会尽量只读取需要的列。列式存储与列裁剪结合时效果尤其明显。

6.3 Constant Folding 等表达式简化

例如某些常量表达式可以在计划阶段直接化简，而不需要对每条数据重复计算。你不必背规则名，但要理解 Catalyst 会做“等价而更省”的改写。

7. Physical Plan：同一个“算什么”，可能有多种“怎么跑”

逻辑计划确定后，Spark 还必须决定具体执行算子。例如一个 Join，从逻辑上都是“按 key 关联”，但物理上可以有不同策略。



物理策略	典型适用情况	最关键代价
BroadcastHashJoin	一侧足够小，可以广播	把小表复制到各 Executor；可避免大表 Shuffle
SortMergeJoin	两侧都较大、等值 Join 常见	两边按 Join Key 重分区并排序，Shuffle 成本高
ShuffledHashJoin	特定规模/配置下可能选择	两侧通常仍需 Shuffle，一侧分区内建 Hash 表

重要：“逻辑 Join”不等于“物理 Join 算法”。你在 explain() 中真正需要找的是 Spark 最终选了哪一种物理策略。

8. explain()：你观察 Catalyst 的窗口

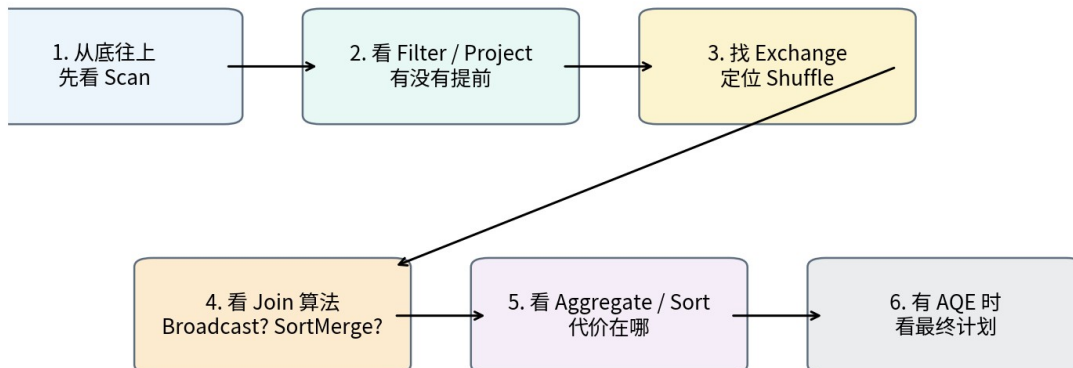
Apache Spark 当前 PySpark DataFrame.explain() 支持多种模式。学习阶段最常用的是 simple、extended、formatted 和 cost。

模式	主要看到什么	适合什么时候
simple	物理计划	快速判断有没有 Shuffle / Join 类型
extended	Parsed、Analyzed、Optimized、Physical 全部计划	学习优化器、对比计划前后变化
formatted	物理计划轮廓 + 每个节点详细信息	计划复杂时更容易读
cost	优化逻辑计划 + 可用统计信息	研究估计行数/大小是否影响决策
codegen	物理计划及可用的生成代码	进阶观察代码生成

```
df.explain()           # 默认: physical plan
df.explain(mode="extended") # 四层计划一起看
df.explain(mode="formatted") # 更适合读复杂 physical plan
df.explain(mode="cost")    # 查看统计/估计信息
```

建议：刚开始不要试图理解 explain() 的每一个内部 ID。先学会识别几个高价值关键词，再逐渐扩展。

看到 explain() 输出时，按这条路线读



不要逐字读计划；先找“数据从哪里来、在哪里过滤、哪里 Shuffle、怎么 Join、怎么聚合”

9. explain() 中最值得认识的 10 个关键词

关键词	通俗解释	看到它时想到什么
Scan / FileScan	从数据源读取	读了哪些列？有没有下推过滤？
Filter	过滤行	是不是尽可能早发生？
Project	选择/计算列	是否只保留真正需要的列？
HashAggregate	哈希聚合	通常用于 groupBy / sum / count 等
Exchange	重新分区/数据交换	高度关注：通常意味着 Shuffle
Sort	排序	可能带来 CPU/内存/磁盘开销
BroadcastExchange	准备广播一份数据	通常是广播 Join 的前置动作
BroadcastHashJoin	广播小表 + 本地 Hash Join	通常避免大表 Shuffle
SortMergeJoin	两边排序后归并 Join	通常看到 Exchange + Sort
AdaptiveSparkPlan	AQE 正在参与	初始计划可能在运行时继续调整

最重要的关键词：如果你只能先学一个，就是 Exchange。它往往对应一次跨 Partition 数据重分布，也就意味着网络、序列化和 Stage 边界。

10. Exchange 为什么几乎是性能分析的第一观察点

为什么执行计划里的 Exchange 值得高度关注？



看到 Exchange，通常要联想到：网络、磁盘、序列化、Stage 边界、数据倾斜风险

例如下面的聚合：

```
df.groupBy('city').sum('amount')
```

原始数据的同一个 city 可能散落在不同 Partition。为了让“北京”的数据在某个下游 Task 中见面，Spark 必须按 city 重新分区，因此物理计划里常见类似：

```
Exchange hashpartitioning(city, N)
+- HashAggregate(...)
+- Scan ...
```

这和前面 MapReduce 学过的 Shuffle 思想本质一致：相同 Key 的记录必须被重组到合适的下游分区。

迁移理解：MapReduce 里你显式学习 Partition + Shuffle；Spark SQL 里这些机制被 Physical Plan 算子包装起来，但底层问题没有消失。

11. Partial Aggregate：Spark 为什么常常先“局部汇总”再 Shuffle

对 SUM、COUNT 等可合并聚合，Spark 通常可以在上游分区先做局部聚合，再把更小的中间状态通过 Shuffle 发送到下游。

```
Partition 0: Beijing 10, Beijing 20 -> Beijing 30
Partition 1: Beijing 50, Beijing 40 -> Beijing 90

Shuffle 后: Beijing -> [30, 90]
Final Aggregate -> 120
```

这与 MapReduce 中 Combiner 的思想有相似之处：尽量减少进入网络的数据量。但具体实现和语义属于 Spark SQL 的聚合执行计划，不要把它简单等同于 Hadoop Combiner API。

12. Join 案例：为什么两句几乎一样的代码可能差很多

```
orders = spark.read.parquet("/data/orders")      # 很大
products = spark.read.parquet("/data/products")  # 很小

result = orders.join(products, "product_id")
result.explain(mode="formatted")
```

如果 Spark 根据统计信息判断 products 足够小，可能选择 BroadcastHashJoin：小表被广播，大表不需要因为 Join Key 做全量 Shuffle。

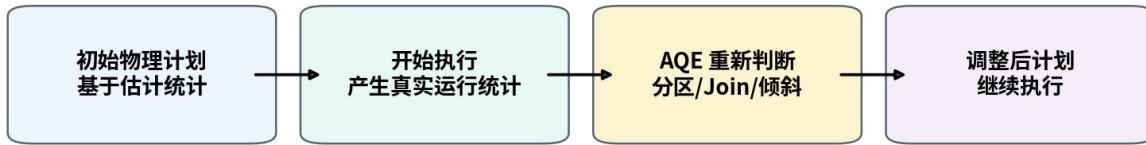
如果两侧都很大，则常见 SortMergeJoin：两侧都需要按 Join Key 建立合适分区，并进行排序/归并。

统计信息为什么重要：优化器的选择依赖“这张表大概多大、过滤后大概还有多少数据”等估计。统计信息缺失或严重失真时，物理计划可能不是你预期的。

当前 Spark 官方文档也明确指出：统计信息会影响执行计划选择；可以通过 `explain(mode="cost")` 观察可用的估计统计。

13. AQE：为什么 Physical Plan 也不一定“一锤定音”

Adaptive Query Execution (AQE)：执行中再优化



典型能力：合并过小 Shuffle 分区 / 处理倾斜分区 / 某些情况下改变 Join 策略

传统思维容易认为：计划一生成就固定不变。但现实中，优化前对数据规模的估计可能不准确。Adaptive Query Execution (AQE) 利用运行时统计信息，在执行过程中进一步调整计划。

常见能力包括：

- 根据实际 Shuffle 输出大小合并过小的下游分区，减少大量微小 Task。
- 识别并处理某些倾斜 Shuffle 分区。
- 在满足条件时，将原本更重的 Join 策略转换为更合适的 Join。

重要区别：Catalyst 的“优化”不只发生在 Job 开始之前；启用 AQE 后，运行时统计可以继续参与决策。

在当前 Spark 官方文档中，AQE 默认启用，并包含合并 Shuffle 分区、倾斜 Join 优化以及某些 Join 策略转换能力。不同 Spark 版本的具体默认参数可能不同，所以生产调优时应查看与你安装版本对应的文档。

14. 从 Physical Plan 到 Stage / Task：最后一公里

Physical Plan 是“算子执行方案”，但集群最终仍然要生成 Stage 和 Task。你可以用下面这条链条理解：

```

SQL / DataFrame
  ↓
Logical Plan
  ↓ Catalyst
Optimized Logical Plan
  ↓ planner
Physical Plan
  ↓ 根据 Exchange / Shuffle 等边界
Stages
  ↓ 每个 Stage 按 Partition 生成
Tasks
  ↓
Executors 执行

```

因此，Physical Plan 中的 Exchange 往往对应 Spark 执行层的 Shuffle 依赖，也通常意味着上下游会处于不同 Stage。

不要机械对应：不是说 explain() 里每一个算子都对应一个 Stage。多个窄依赖算子往往会被放在同一个 Stage / pipeline 中执行。

15. 一个完整实验：自己观察优化前后计划

```

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("PlanDemo").getOrCreate()

data = [
    (1, "Beijing", "paid", 100.0),
    (2, "Beijing", "cancel", 80.0),
    (3, "Shanghai", "paid", 120.0),
    (4, "Beijing", "paid", 200.0),
]

df = spark.createDataFrame(data, ["id", "city", "status", "amount"])

q = (df
     .filter(F.col("status") == "paid")
     .select("city", "amount")
     .groupBy("city")
     .agg(F.sum("amount").alias("sales")))

q.explain(mode="extended")
q.show()

```

观察时不要只看最终结果，重点回答：

1. Optimized Logical Plan 中 Filter 和 Project 在什么位置？
2. Physical Plan 中有没有 HashAggregate？
3. 有没有 Exchange？它为什么出现？
4. Exchange 的分区 Key 是什么？
5. 最终用了几个聚合阶段？

16. 第二个实验：观察 Broadcast Join 与普通 Join

```
from pyspark.sql import functions as F

orders = spark.createDataFrame([
    (1, 101, 100.0), (2, 102, 200.0), (3, 101, 50.0)
], ["order_id", "product_id", "amount"])

products = spark.createDataFrame([
    (101, "Phone"), (102, "Laptop")
], ["product_id", "product_name"])

# 普通写法：让 Spark 自己选择
q1 = orders.join(products, "product_id")
q1.explain(mode="formatted")

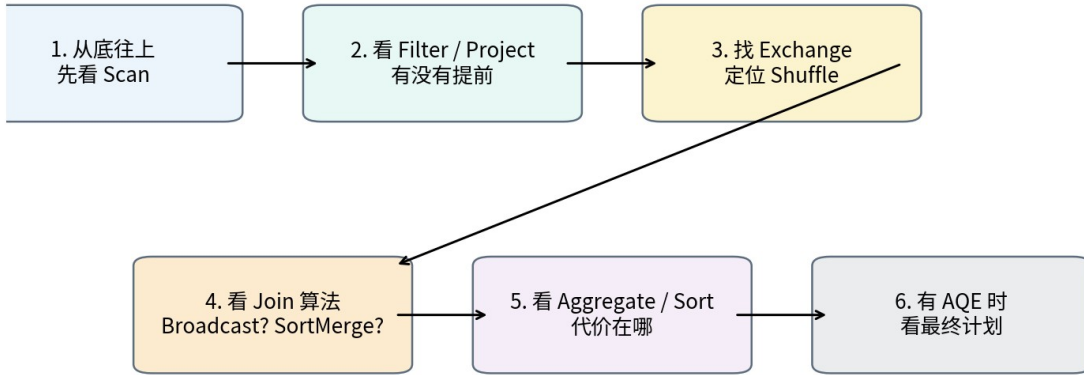
# 显式广播提示（教学观察用）
q2 = orders.join(F.broadcast(products), "product_id")
q2.explain(mode="formatted")
```

比较两份计划，重点找：BroadcastExchange、BroadcastHashJoin、Exchange、SortMergeJoin。实际选择会受到统计信息、数据规模、配置和 Spark 版本影响。

不要把 Hint 当命令：Join Hint 是给优化器的重要偏好信息，但并不是所有 Join 类型、所有条件下都能强制采用某一种策略。

17. 一个实用的 explain() 阅读顺序

看到 explain() 输出时，按这条路线读



不要逐字读计划；先找“数据从哪里来、在哪里过滤、哪里 Shuffle、怎么 Join、怎么聚合”

面对几十行甚至上百行 Physical Plan，初学者最容易犯的错误是从第一行开始逐字读。更有效的方法是带着问题读：

问题	重点关键词	你想判断什么
数据从哪里来？	Scan / FileScan	读什么格式、哪些列、哪些过滤被下推
哪里发生网络重分布？	Exchange	Shuffle 次数、分区字段、潜在倾斜
Join 怎么做？	BroadcastHashJoin / SortMergeJoin	是不是广播小表，还是两边 Shuffle
哪里聚合？	HashAggregate	局部/最终聚合结构
是否有排序？	Sort	为什么排序、是否由 Join/OrderBy 引起
是否启用运行时调整？	AdaptiveSparkPlan	AQE 是否参与

最低闭环：你不需要一开始看懂每个内部节点。只要能回答“Scan -> Filter/Project -> Exchange -> Join/Aggregate -> 输出”这条主链，就已经具备第一层执行计划分析能力。

18. 常见误区与纠正

误区 1: Catalyst 会自动把任何程序变快。

纠正：它主要优化结构化 SQL/DataFrame 表达式；黑盒 Python UDF 等会限制优化空间。

误区 2: 没有报错就说明计划很好。

纠正：正确性和性能是两回事；一个正确查询仍可能因为 Shuffle、Join 选择或倾斜而很慢。

误区 3: 看到 SortMergeJoin 就说明 Spark 选错了。

纠正：两边都很大时它可能是合理选择；必须结合数据规模、统计和运行时指标判断。

误区 4: Broadcast Join 永远最快。

纠正：广播对象过大可能造成网络 and Executor 内存压力。

误区 5: Exchange 就是错误。

纠正：Shuffle 是很多分组、Join、重分区问题的必要代价，目标是减少不必要 Shuffle，而不是消灭所有 Shuffle。

误区 6: explain() 就等于 Spark UI。

纠正：explain 更偏计划；Spark UI 提供实际运行指标。两者需要结合。

误区 7: Physical Plan 一开始就永远不变。

纠正：AQE 可以根据运行时统计调整部分执行策略。

19. 练习题

6. 为什么 RDD 中的 lambda 比 DataFrame 的结构化表达式更难被 Catalyst 优化？
7. ``groupBy("city").sum("amount")`` 为什么通常会出现 Exchange？
8. 一张 5 MB 维表与 1 TB 事实表做等值 Join，哪类 Join 值得优先观察？为什么？
9. 如果 explain() 中出现 ``Exchange hashpartitioning(user_id, 200)``，你至少能推断出什么？
10. 为什么 ``explain(mode="cost")`` 中的统计信息可能影响 Join 选择？
11. AQE 与“查询开始前的优化”有什么关键差别？

20. 参考答案

1. RDD lambda 对 Spark 来说接近用户自定义黑盒函数；DataFrame 表达式显式暴露列、运算符和关系，因此可以被规则分析和改写。
2. 因为相同 city 的记录可能分布在多个上游 Partition；最终聚合需要让同一 city 的中间状态到合适的下游 Partition 汇合。

3. BroadcastHashJoin。若小表确实足够小，将它广播到 Executor 可让大表避免按 Join Key 全量 Shuffle。
4. 数据将按 user_id 做一次重新分区；通常意味着 Shuffle，并很可能形成 Stage 边界。200 是计划中的目标 Shuffle 分区数或相关分区表达。
5. 优化器需要估计各节点输出规模，Join 两侧大小会影响是否适合广播以及其他物理策略选择；错误统计可能导致不理想计划。
6. AQE 在真正运行后利用实际统计继续调整，例如合并 Shuffle 分区、处理某些倾斜情况或调整部分 Join 策略。

21. 本讲最终知识地图



一句话总结：Catalyst 的本质，是把“用户描述的计算目标”逐步转换成“可在分布式集群上执行的具体计划”，并在等价前提下尽量减少不必要的数据读取、传输和计算。

与 MapReduce 的连接：MapReduce 时代很多执行结构需要程序员更显式地设计；Spark SQL 把更多决策交给优化器，但 Shuffle、Partition、数据倾斜、网络与磁盘成本依然存在。

22. 下一讲预告

第 17 讲将进入 Spark on YARN 与资源模型：Driver、ApplicationMaster、Executor、Container、Core、Memory 到底如何对应；执行一个 spark-submit 后 YARN 集群内部发生什么；以及为什么“Executor 越多越快”“内存越大越快”都是过度简化。

参考资料

1. Apache Spark 4.2.0 Documentation - PySpark DataFrame.explain：解释 simple、extended、codegen、cost、formatted 等模式。
2. Apache Spark 4.2.0 Documentation - Spark SQL Performance Tuning：统计信息、自动广播 Join、Join Hints、Adaptive Query Execution 等。

说明：本讲以稳定的 Spark SQL/Catalyst 原理为主；具体默认参数、算子展示形式会随 Spark 版本变化，实际实验应以安装版本的官方文档和 explain() 输出为准。